

EJERCICIOS

El objetivo es practicar el control de secuencias, acumuladores y series numéricas.

1. Suma de Cuadrados: Pedir un número N y calcular la suma de los cuadrados de los números desde 1 hasta N ($1^2 + 2^2 + \dots + N^2$).

C++:

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
```

```
    int n;
```

```
    int suma = 0; // Acumulador
```

```
    cout << "Ingresa un numero N: ";
```

```
    cin >> n;
```

```
    // Desde i=1; mientras i sea menor o igual a N; suma 1 a i cada vuelta
```

```
    for (int i = 1; i <= n; i++) {
```

```
        suma = suma + (i * i); // En C++ puedes multiplicar i * i para el cuadrado
```

```
    }
```

```
    cout << "La suma de los cuadrados es: " << suma << endl;
```

```
    return 0;
```

```
}
```

Python:

```
# Lee el limite N ingresado por el usuario
```

```

n = int(input("Ingresa un numero N: "))
suma = 0 # Crea un acumulador y lo inicia en 0

# Genera numeros desde el 1 hasta N (se pone n+1 porque range omite el ultimo valor)
for i in range(1, n + 1):
    suma = suma + (i ** 2) # Suma el cuadrado del numero actual al acumulador

print("La suma de los cuadrados es:", suma)

```

2. Tabla de Multiplicar Inversa: Mostrar la tabla de multiplicar de un número dado, pero desde el 10 hasta el 1 (ej: 5×10, 5×9 ...)

C++:

```

#include <iostream>

using namespace std;

int main() {
    int numero;

    cout << "Ingresa un numero para ver su tabla inversa: ";
    cin >> numero;

    cout << "Tabla del " << numero << " al revés:" << endl;

    // Desde i=10; mientras i sea mayor o igual a 1; resta 1 a i cada vuelta
    for (int i = 10; i >= 1; i--) {
        cout << numero << " x " << i << " = " << (numero * i) << endl;
    }

    return 0;
}

```

```
}
```

Python:

```
# Lee el numero para calcular su tabla
```

```
numero = int(input("Ingresa un numero para ver su tabla inversa: "))
```

```
print("Tabla del", numero, "al reves:")
```

```
# range(10, 0, -1) significa: empieza en 10, llega hasta el 1, restando 1 paso a la vez
```

```
for i in range(10, 0, -1):
```

```
    # La letra 'f' antes de las comillas permite meter variables directo en el texto con llaves {}
```

```
    print(f"{numero} x {i} = {numero * i}")
```

3. Factorial de un número: Calcular el producto de todos los números desde 1 hasta N.

C++:

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
```

```
    int n;
```

```
    int factorial = 1; // Acumulador de multiplicacion empieza en 1
```

```
    cout << "Ingresa un numero N para calcular su factorial: ";
```

```
    cin >> n;
```

```
    for (int i = 1; i <= n; i++) {
```

```
        factorial = factorial * i;
```

```

    }

    cout << "El factorial de " << n << " es: " << factorial << endl;

    return 0;
}

```

Python:

```

# Lee el numero N ingresado por el usuario
n = int(input("Ingresa un numero N para calcular su factorial: "))
factorial = 1 # Acumulador de multiplicacion (inicia en 1)

# Recorre los numeros desde el 1 hasta N
for i in range(1, n + 1):
    factorial = factorial * i # Multiplica el valor actual por el acumulador

print("El factorial de", n, "es:", factorial)

```

4. Dibujo de Rectángulo: Pedir un ancho y un alto, y dibujar un rectángulo sólido usando el carácter “*”. (Requiere un for dentro de otro).

C++:

```

#include <iostream>

using namespace std;

int main() {
    int ancho, alto;

    cout << "Ingresa el ancho del rectangulo: ";
    cin >> ancho;

    cout << "Ingresa el alto del rectangulo: ";

```

```

cin >> alto;

// Bucle para el alto (filas)
for (int i = 0; i < alto; i++) {
    // Bucle anidado para el ancho (columnas)
    for (int j = 0; j < ancho; j++) {
        cout << "*"; // Imprime sin saltar de linea
    }
    cout << endl; // Salto de linea al terminar la fila
}

return 0;
}

```

Python:

```

# Pedimos el ancho y el alto al usuario
ancho = int(input("Ingresa el ancho del rectángulo: "))
alto = int(input("Ingresa el alto del rectángulo: "))

# Bucle externo: controla las filas (el alto)
for i in range(alto):
    # Bucle interno: dibuja los asteriscos de una sola fila (el ancho)
    for j in range(ancho):
        print("*", end="") # end="" evita que salte a la siguiente línea

    # Al terminar de dibujar la fila completa, hacemos un print vacío para saltar de línea
    print()

```

5. Cálculo de Potencia: Pedir la base y el exponente, y calcular el resultado realizando multiplicaciones sucesivas (sin usar funciones pow o el operador **).

C++:

```
#include <iostream>

using namespace std;

int main() {
    int base, exponente;
    int resultado = 1;

    cout << "Ingresa la base: ";
    cin >> base;
    cout << "Ingresa el exponente: ";
    cin >> exponente;

    for (int i = 0; i < exponente; i++) {
        resultado = resultado * base;
    }

    cout << "El resultado de la potencia es: " << resultado << endl;
    return 0;
}
```

Python:

```
# Pedimos los datos

base = int(input("Ingresa la base: "))
exponente = int(input("Ingresa el exponente: "))
```

```
# Empezamos el resultado en 1 porque es una multiplicación
resultado = 1

# Repetimos la acción tantas veces como nos indique el exponente
for i in range(exponente):
    resultado = resultado * base # Multiplica el valor actual por la base

print("El resultado de la potencia es:", resultado)
```

6. Filtro de Pares e Impares: Leer 10 números por teclado. Al finalizar, mostrar cuánto sumaron solo los números pares y cuántos números impares se ingresaron.

C++:

```
#include <iostream>

using namespace std;

int main() {
    int numero;

    int suma_pares = 0;
    int contador_impares = 0;

    cout << "Por favor, ingresa 10 numeros:" << endl;

    for (int i = 0; i < 10; i++) {
        cout << "Ingresa el numero " << i + 1 << ": ";
        cin >> numero;

        if (numero % 2 == 0) {
            suma_pares = suma_pares + numero; // Acumula si es par
```

```

    } else {
        contador_impares++; // Suma 1 al contador si es impar (es igual a hacer + 1)
    }
}

cout << "La suma total de los numeros pares es: " << suma_pares << endl;

cout << "La cantidad de numeros impares ingresados fue: " << contador_impares <<
endl;

return 0;
}

```

Python:

```

suma_pares = 0 # Acumulador: guardará la suma de los valores
contador_impares = 0 # Contador: solo cuenta cuántos hay (suma de 1 en 1)

print("Por favor, ingresa 10 números:")

# El bucle se repetirá exactamente 10 veces
for i in range(10):
    numero = int(input(f"Ingresa el número {i + 1}: "))

    # Verificamos si el número es par usando el módulo (%)
    if numero % 2 == 0:
        suma_pares = suma_pares + numero # Sumamos el valor del número
    else:
        # Si no es par, por lógica es impar
        contador_impares = contador_impares + 1 # Solo sumamos 1 al conteo

# Imprimimos los resultados finales fuera del bucle

```

```
print("La suma total de los números pares es:", suma_pares)
print("La cantidad de números impares ingresados fue:", contador_impares)
```

7. Buscador de Divisores y Primalidad: Dado un número N, mostrar todos sus divisores. (Opcional: Si solo tiene dos divisores, imprimir que el número es primo).

C++:

```
#include <iostream>
using namespace std;

int main() {
    int n;
    int contador_divisores = 0;

    cout << "Ingresa un numero N: ";
    cin >> n;

    cout << "Los divisores de " << n << " son:" << endl;

    for (int i = 1; i <= n; i++) {
        if (n % i == 0) {
            cout << i << endl;
            contador_divisores++; // Incrementamos el contador
        }
    }

    if (contador_divisores == 2) {
        cout << "¡El numero " << n << " es primo!" << endl;
    }
}
```

```
    return 0;
}
```

Python:

```
# Pedimos el número al usuario
```

```
n = int(input("Ingresa un número N: "))
```

```
contador_divisores = 0 # Usamos un contador para la parte opcional
```

```
print(f"Los divisores de {n} son:")
```

```
# Recorremos todos los números desde el 1 hasta N
```

```
for i in range(1, n + 1):
```

```
    # Si la división es exacta, 'i' es un divisor
```

```
    if n % i == 0:
```

```
        print(i)
```

```
        contador_divisores = contador_divisores + 1 # Contamos que hallamos un divisor
```

```
# Parte opcional: Verificamos si es primo
```

```
if contador_divisores == 2:
```

```
    print(f"¡El número {n} es primo!")
```

8. El Juego de TrikiTraka: Imprimir los números del 1 al 50, pero con estas reglas:

o Si el número es múltiplo de 3, imprime "Triki".

o Si es múltiplo de 5, imprime "Traka".

o Si es múltiplo de ambos (3 y 5), imprime " TrikiTraka".

o Si no es ninguno, imprime el número.

C++:

```
#include <iostream>

using namespace std;

int main() {

    cout << "Iniciando el juego TrikiTraka:" << endl;

    for (int i = 1; i <= 50; i++) {

        // En C++ usamos && para decir "Y" (and)
        if (i % 3 == 0 && i % 5 == 0) {

            cout << "TrikiTraka" << endl;

        } else if (i % 3 == 0) {

            cout << "Triki" << endl;

        } else if (i % 5 == 0) {

            cout << "Traka" << endl;

        } else {

            cout << i << endl;

        }

    }

    return 0;

}
```

Python:

```
print("Iniciando el juego TrikiTraka:")

# Recorremos del 1 al 50 (ponemos 51 porque el range no incluye el último número)
for i in range(1, 51):
```

PRIMERO: Verificamos si es múltiplo de 3 Y de 5 (también podrías usar $i \% 15 == 0$)

if $i \% 3 == 0$ and $i \% 5 == 0$:

print("TrikiTraka")

SEGUNDO: Si no cumplió la de arriba, vemos si es solo de 3

elif $i \% 3 == 0$:

print("Triki")

TERCERO: Si no cumplió las anteriores, vemos si es solo de 5

elif $i \% 5 == 0$:

print("Traka")

CUARTO: Si no es múltiplo de ninguno, solo imprimimos el número

else:

print(i)